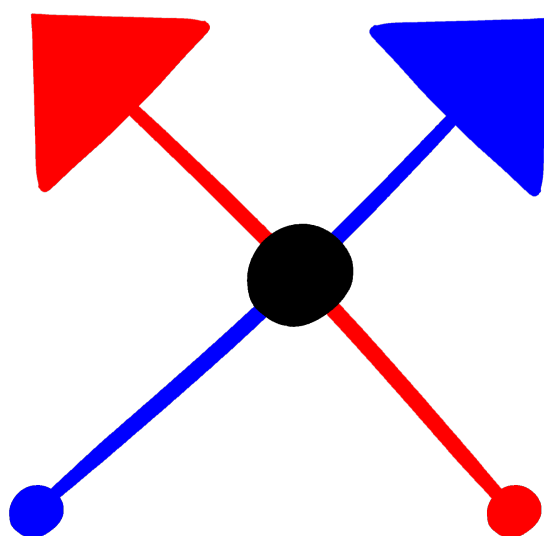


**UNIVERSITAT
JAUME I**

VIDEO GAME DESIGN AND DEVELOPMENT



Tactical Vector

Development of a Math Serious Game for Mobile Devices

Author

Joel Aguilera Molina

Supervised by

Ignacio Miralles Tena

UNIVERSITAT JAUME I

11th of July, 2025

Aknowledgments

To my father and mother who are always there supporting me. To my supervisor for still giving me very useful lessons and worrying about my project even knowing that he has no obligation to do so, and lastly to my close friends and classmates for their unconditional support.

Abstract:

Tactical Vector is a strategy, turn-based video game, with the objective of being a math serious game for high school students, having a great emphasis on the calculation and visual logic of functions in the real coordinate plane.

In the video game, a main character will be controlled. This avatar will be progressing through a sequence of levels, where in all of them, it will have to fight different enemies, with specific math problems to solve and different themes following a progression related to the math level studied in high school in Spain, progressing from the first to last topics learned around the academic year.

In this project, it will be described the process and details about the development of the project, conceptualization of the idea and technical application of the different mechanics, highlighting the mathematical formulas as a tool for the player to progress.

Academically, this document consists of the final degree project report of the Video game Design and Development bachelor's degree at the Jaume I University

Keywords: Android, Serious Game, Educational, Math.

Contents

1	Technical Proposal	1
1.1	Introduction	1
1.2	Work Motivation	2
1.3	Related Subjects	3
1.4	Objectives	4
1.5	Planning	5
1.6	Expected Results	6
1.7	Tools	6
2	Game Design Document	7
2.1	Introduction	7
2.1.1	Game Title	7
2.1.2	Summary	7
2.1.3	Objective of the game	7
2.1.4	Platforms	8
2.1.5	Target Audience	8
2.2	Game Mechanics	9
2.2.1	Player Control	9
2.2.2	Game Rules	11
2.2.3	Game System	11
2.2.4	Interaction with NPCs	12
2.2.5	Physics and Game Logic	12
2.3	Narrative and Story	13
2.3.1	General synopsis	13
2.3.2	Main Characters	13
2.3.3	History and background of the world	14
2.3.4	Missions or story progression	14
2.4	Level Design	14
2.4.1	Overview of the scenarios	14
2.4.2	World Map	14
2.4.3	Challenges	14
2.4.4	Difficulty curve	15
2.5	Art and visual style	16
2.5.1	Artistic direction	16
2.5.2	Scenery and enviorement	17
2.5.3	UI y UX	17
2.6	Music and Sound	18

2.6.1	Soundtrack style	18
2.6.2	Key sound effects	18
2.7	Technical aspects	19
2.7.1	Engine	19
2.7.2	Programming languages	19
2.7.3	Minimal requisites and recommended	19
2.7.4	AI and NPC behavior	20
3	Technical implementation in Unity	21
3.1	Introduction	21
3.2	Gameloop, UI y UX	21
3.2.1	Fuera de Combate	21
	Menú Principal	21
	Map	22
3.2.2	In Combat	22
	Attack selection	22
3.3	Turn System	23
3.3.1	General Description	23
3.3.2	Turn Controller	23
3.3.3	UI Integration	24
3.4	Movement System and vectors	26
3.4.1	Mathematical fundament	26
	Cards	26
	Applciation in code	27
3.4.2	Map Movement	30
3.4.3	Camera Movement	31
3.5	Combat and Math Problems	31
3.5.1	Exercise-based attacks	31
3.5.2	Problem Generator	31
3.6	AI Intelligence	31
3.7	Level Design	32
3.7.1	Creating and Loading Levels	32
3.8	Save system	33
3.9	Sound and Music	33
4	Results	35
5	Conclusions and Future Lines of Work	37
	Bibliography	39
	Appendix A Link to the code repository	43

CHAPTER 1

Technical Proposal

1.1 Introduction

It was planned to develop a mobile game with twelve levels that will be unlocked one after another in a similar fashion as Angry Birds. To understand the mechanics, the game will present an optional tutorial on the first level where the player can experiment how the problems are presented and how to solve them.

In the paper it will be detailed the assets, art and music created and downloaded for the development of the game, the programming required for the game to be in a playable state and with the mechanics correctly applied (specially for the function display and its representation), and the justification and explanation of the balance between the ludic and the formative aspect of the game and how the players will be gaining the intention of solving problems and indirectly learn math thanks to the design choices.

1.2 Work Motivation

The Trends in International Mathematics and Science Study (TIMSS) ¹ published a report of data in 2023 about the resolution of cognitive competencies in math. This report showed that our level is 498 points, a level inferior to the average total of the European Union (513 points); it is a notably low number which has been slightly lowering since 2015 ².

With the objective of being able to raise the social awareness of learning math, several countries that we compare with from the European Union have been using resources on digital tools and innovative methodologies of teaching, which have affected positively the general results of the students.

A research about the impact of math serious games in the studentship here in Spain transpired in a school of Galicia with students going from first to fourth year of studies, with results that showed that the students who followed the program of playing the serious games showed a remarkable level on understanding new lessons about math and higher grades than usual on the subject ³.

The development of this game is proposed as an extension to expand the range of participants to the high school students. With the intention of giving motivation in their learning and indirectly refining their abilities, similarly to the research mentioned before, with the objective of giving the youth a more ludic way of approaching math.

¹Bibliography 1, [de Evaluación Educativa n.d.\(a\)](#)

²Bibliography 2, [de Evaluación Educativa n.d.\(b\)](#)

³Bibliography 3, [Fraga Varela n.d.](#)

1.3 Related Subjects

- **Mathematics II - VJ1211** . While Mathematics I could be noted as a great base for developing the material that is going to be developed in the game, it was considered that the material and study program of Mathematics II is more useful to back up part of the mechanics.
- **Game Engines - VJ1227**. Since Unity was the game engine used for the development of the game, the materials and theory resources from this subject will be used to work and navigate around the engine.
- **Video Game Art - VJ1223**. The material and lessons learned from the subject will help to create and design the HUD and 2D elements of the game.
- **Mobile Device Applications - VJ1229**. The documentation and programming about the way of presenting and managing the interactive elements in mobile and understand the execution mode of it will be useful to help develop the gameflow and state managing of the game.
- **Foundations for Educational Video Game Design - VJ1238**. While the student have taken the subject on Erasmus, being Serious Game Design, this actually was one of the main motivators for the development of a Serious Game project, thanks to the insightful experience abroad and, the techniques, balance between formative and entertainment, and practice on other serious game projects will be very helpful in the conceptualization and design choices of the project.

1.4 Objectives

- **Develop and share a playable prototype:** A game that is enjoyable from start to finish without serious errors that make the experience unplayable and teaches the game mechanics in the intended way.
- **Creating and maintaining a specific artistic style:** orthographic low-poly 3D top view with a simplified aesthetics but recognizable, the 2D elements are inspired on the strokes of the pen and pencils on a notepad, giving the idea that the student is creating the world.
- **Creating and developing at least 10 levels:** All of them will have different themes of problems and math solving with the objective of making every level unique and with different learning requirements.
- **Apply mathematics as a mechanic needed to complete levels:** The game demands you to know how to multiply, divide, sum, simplify formulas... As a way of defeating a foe or resolving part of a puzzle for each level of the game.
- **Integrating genres and ludic elements naturally on par with mathematics:** Applying a card system, completion awards, rewards for beating levels... Which help the user to not feel burnt out for the math side but also making them not being the principal focus of attention and lose the objective of the game which is educational.
- **Publishing the game:** Make the game available to download as an Application for the public to download as a 1.0 version, with everything that it takes in consideration (logo, description, publishing, creation of study if necessary...).

1.5 Planning

Task	Description	Hours
Art: Main Character Model	Creating the model of the main character	8
Art: NPCs Model	Creating the model of the NPCs	10
Art: Environment Models	Creating the models around the map	14
Art: Main Character Animations	Creating of all the animations of the main character for all levels	13
Art: NPC Animations	Animations of the NPCs that will appear in different levels	10
Art: Map	Developing the main map of the game with all the levels	10
Art: UI	Creating the 2D elements, background, user interface, hud...	15
Art: Logo	Logo design	5
Programming: Touch	Developing touch interaction of the game	6
Programming: Tutorial	Creating the tutorial and its events	14
Programming: Menus	Developing title screen, map menu and the deck manager	10
Programming: Options	Developing the options of the application	6
Programming: Save System	Developing save system	3
Programming: Combat	Developing combat system	29
Programming: Levels	Developing and programming all levels of the game	20
Documentation: Technical Proposal	Development of technical proposal and updates	10
Documentation: Research	Research of the theme, level that is used of math, search of exercises...	24
Documentation: Game Design Document	Redacting GDD	30
Documentation: Final Degree Project	Redacting final degree project	45
Documentation: Defense Video	Developing the defense video of the final degree project	8
Design: Applying Research	Synthesize all the research into level design	10
Design: Music	Finding music and sound effects	5

Task	Description	Hours
Art: Sum of Hours	Hours used in art	75
Programming: Sum of Hours	Hours used in programming	93
Documentation: Sum of Hours	Hours used in documentation	117
Design: Sum of Hours	Hours used in design	15
Total Hours		300

1.6 Expected Results

Upon finalizing this project, it is expected to develop a prototype application of mobile for Android users, compatible with almost every device. This application will apply ludically mathematics as a mechanic with the objective of lowering the general anguish of students that provokes the math lessons for the high school level in Spain.

1.7 Tools

- **Unity Engine:** Specifically Unity Engine 6000.0.45f1, the engine where the game will be developed.
- **Krita:** Program that will be used for image editing and 2D asset creation.
- **Visual Studio Code:** For programming the scripts inside of the game.
- **Overleaf:** For the design and creation of the memory of the final degree.
- **Unity Asset Store:** For the use of external assets, which will be linked in the Bibliography.
- **Blender:** For editing and transforming extensions of 3D assets and creating simple ones.
- **Adobe Audition:** For audio editing.
- **OpenAI:** For general purposes such as research finding, support on the memory redaction and solving simple problems.
- **SketchFab:** For finding external 3D models, which will be linked in the Bibliography.
- **Pixabay:** For finding external music themes, which will be linked in the Bibliography.
- **Freesound:** For finding external sound effects, which will be linked in the Bibliography.

CHAPTER 2

Game Design Document

2.1 Introduction

2.1.1 Game Title

Tactical Vector.

2.1.2 Summary

Tactical Vector its a turn based strategy game of 3D top view where the player will have to beat a series of levels by controlling Descartes, the avatar. In these levels the player will fight in a map filled with obstacles and enemies, via using functions on the real plane, the player will have to avoid the obstacles and attack the enemies via resolving problems to advance between levels to defeat Thue-Morse, the last enemy and main villain of the game.

2.1.3 Objective of the game

Most of the serious math-themed games that have been created tend to cover elements at a level much lower than high school (middle school, sums...), and those that do treat the high school level, put a lot of emphasis on math, so the "game" element is lost in favor of more of an application of solving math problems.

Tactical Vector proposes a game that inherently relies on strategic ability and the ability to plan movements, positioning, and resource management, as would any other strategy game on the market. However, its mechanics and values are influenced by the mathematical theme. This means that players are more rewarded for their tactical ability in the game, but to maximize the potential (damage, positioning, etc.) of their strategies, they must learn or utilize mechanics that apply mathematics.

2.1.4 Platforms

The game is designed to be played exclusively on mobile devices, specifically Android.

2.1.5 Target Audience

Due to the very nature of the project's objective as a serious game, the target audience is primarily teenagers (12-16), both men and women with a casual mobile gamer profile. However, thanks to the simplistic design choice and themes, target audience can also be broader, reaching generally people that want to learn math with an entertaining aspect at a solid level. They don't necessarily need to have experience with strategy games, as the mechanics will be simplified, explained and easy to understand for anyone with at least some experience with video games in general.

2.2 Game Mechanics

2.2.1 Player Control

The player will be able to act with an indefinite time margin each time it is his turn to act, which will be divided into the following stages:

- **Movement Phase:** The movement of the game is one of the differentiating mechanics in reference to other games of the same genre: Given a maximum range defined as a statistic for each player, players must calculate a function from their local coordinate axes, in the form $y = ax + b$, $f(x)$ or $y =$ will be immovable and, from a system of cards, the right side of the function can be defined, that is, cards like x , $\sin(x)$, -2 , $+3x$, $+2 \dots$ will come out that when used will give the resulting function that will generate two movement possibilities, there the player will choose the one that best suits him.

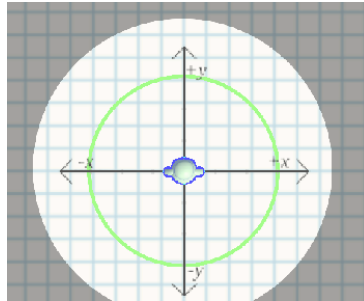


Figure 2.1: Maximum distance and local coordinate axis of the player.

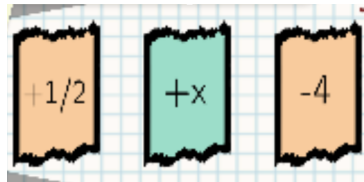


Figure 2.2: Cards to calculate the movement function, 5 cards in hand maximum.

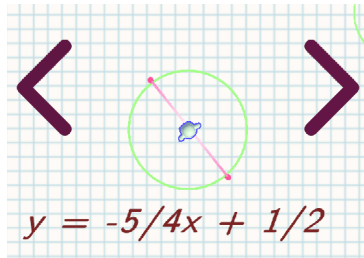


Figure 2.3: Point-to-point selection.

If you collide with an obstacle and choose that path, you will end up in the position where the collision occurred. Enemies also count as obstacles (you cannot pass through them)..

- **Attack Phase** The player's attack style is also very similar to their movement. The movement radius becomes a radius of influence, and within it, you must calculate a function that connects with the enemy to damage it. If this trajectory connects with an obstacle, it will be cut off as if it were a movement. The cards to use are the same. After connecting, depending on your class and who you're facing, you'll get a random multiple-choice problem. Depending on how many attempts you get right, your damage will be increased or decreased.

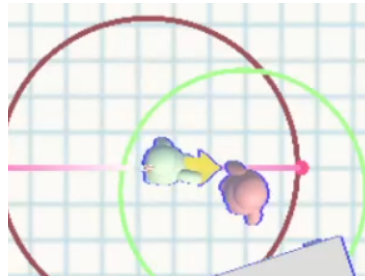


Figure 2.4: Impact of the attack on an enemy.

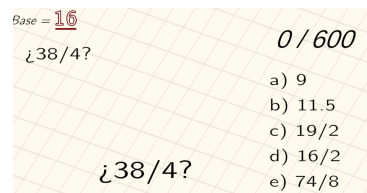


Figure 2.5: Example of a problem to solve to maximize damage.

2.2.2 Game Rules

The player completes a map when all enemies are downed at the start of their turn, and loses it if the player reaches the point of having all their health lost due to an enemy attack, forcing the player to replay the level. All enemies and the player move in a turn-based hierarchy, generally consisting of T.Player , T.Enemy 1 , T.Enemy 2 , ... , T.Enemy Final , T.Player . Repeating, in addition to the aforementioned player control, the map boundaries count as obstacles.

For each level completed, new levels are accessed until the final level is reached, where defeating the final boss will end the game.

2.2.3 Game System

The essence of the game is based on combat. At each level, the player will have to face an undetermined number of enemies within a map boundary (or grid), with certain obstacles around the map that will enhance the player's positioning ability so as not to be pressured by the number of enemies.

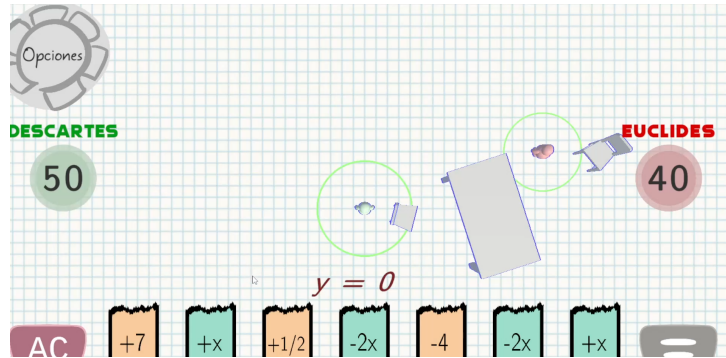


Figure 2.6: Example of a level of Tactical Vector.

There's no direct progression in terms of stat changes, however, with each level, you gain certain cards that you can add to any class's deck. These cards can be used to expand your repertoire of paths, making them increasingly precise and accurate. Aside from this, there are certain levels that don't help you advance toward the final boss (optional paths). Upon completion, they give you a statistical boost (more health or a larger maximum radius) to help simplify your journey.

Looking at the figure below, the level marked at the top gives a maximum radius upgrade, and the level marked at the bottom gives a maximum health upgrade.

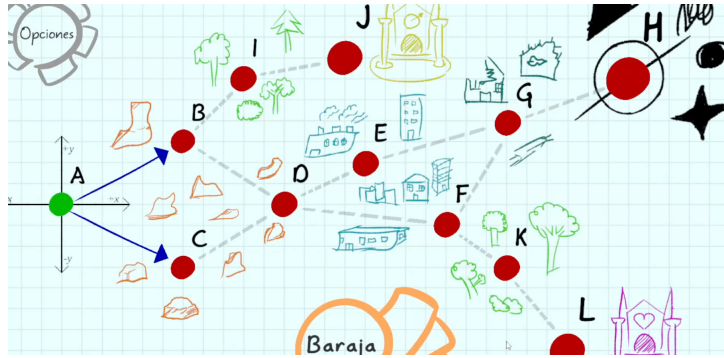


Figure 2.7: Tree-shaped level map.

2.2.4 Interaction with NPCs

Interaction with NPCs will be applied in battles where they will follow the same rules of movement, attack and shuffle turns as the player, only that they will always move after the player on the first turn, if the enemies land a hit on you however, you will also have to solve an exercise from their problem pile to reduce the hit, if you land a hit on the first try, you completely avoid the damage.

2.2.5 Physics and Game Logic

Calculations for movement, impact, and changes in scenes or stages of the game will be calculated using the engine used (Unity 6000.0.45f1), following a mathematical logic similar to what the player has calculated (a straight path, similar to a curved one, etc.).

Note above all that collisions will be key to taking into account the completion of trajectories of certain formulas.

2.3 Narrative and Story

2.3.1 General synopsis

Thue-Morse, the recent emperor of Real Coordinate Space, attempts to carry out, through his power, a plan to return all existence in this world to its origin, the empty set. He believes that mathematics is inherently flawed, that the first axioms are nothing more than reasons for continued existence, and that we should reject all the lies we have accepted and return to the purest and simplest total truth, 0.

However, a person, driven by the aspiration to learn, to expand, to understand the world around him, seeks to stop his plans. He will attempt to put an end to the villain's nihilism to make the population see that mathematics, although imperfect, is the greatest tool for understanding everything around them.

2.3.2 Main Characters

Thue-Morse: The ruthless Thue-Morse has seen that, over time, the complexity and excuses developed to try to keep the original mathematical conventions afloat have become delusions of such absurd difficulty that he truly sees no point in continuing to believe in them. He intends to do away with all mathematics and return to a simpler, more real time, when nothing was written, the empty set.

Descartes: A fervent lover of language, he uses very complex words and knows slang terms that are not his point of interest. He created this resistance because of his love of knowing, of interpreting what surrounds him and being able to simplify and summarize it in words, in ideas that are readable to him and to everyone. And he will not allow his desire for language, his desire to learn, to be denied.

2.3.3 History and background of the world

The Real Coordinate Space is defined as a world in which its inhabitants do not need to eat, sleep or breathe, they are "ideal scenarios" as they are posed in physics, circumstances of worlds of ideas, in them live "humans", they have an anthropomorphic but simplified form, easy to understand and without many variations, but what gives life to their world are not their features, but their tools, their discoveries, their fields of study, they live by and to learn and develop mathematics in a white space marked with an almost invisible gray mesh. The existence of this is basically the sum of all human interpretations of mathematics, which is why the names of these "conceptual humans" refer to fathers of the fields, for example Diophantus is the father of algebra and Thue-Morse comes from the Thue-Morse set or also known as the set of evil numbers.

2.3.4 Missions or story progression

In the story, at certain levels (such as optional upgrade levels or mandatory levels), enemies or sub-bosses will appear with different names and dialogue depending on the class you choose. At the end of the story, you will have a fight with Thue-Morse, with dialogue at the beginning and end that will mark the end of the game.

2.4 Level Design

2.4.1 Overview of the scenarios

As previously mentioned, all levels follow a map border, the original position of the player and enemies, and obstacles around the map. These provide a tactical advantage for the player to block enemy attacks or movement paths. However, they also make it difficult to land attacks if you're not in the right position. See Figure 2.6 for a visual representation.

2.4.2 World Map

The game consists of 12 levels, with the first level being the game's tutorial and the last being the final boss. The world is navigated by completing a level, unlocking the branches attached to it on its right. To reach the final level, therefore, it is necessary to complete at least 5 levels before reaching the final level. However, as previously mentioned, completing more levels will give the player more resources to expand their arsenal and their capabilities against the final boss.

See Figure 2.6 for a visual representation.

2.4.3 Challenges

While the maps try not to be too complex in their environment (a forest with few trees, a rocky desert, a city with a few buildings and a few streetlights), the difficulty of the enemies is defined by their health, the damage they deal, which gradually increases and requires you to think harder and harder to solve the levels, making fewer and fewer mistakes.

The challenges or mini-bosses are a small collection of topics covered in the previous levels. The first one above will cover functions, algebra, and equations, while the one below will cover statistics, and probability.

2.4.4 Difficulty curve

To begin, the tutorial level will use numbers and straightforward problems (calculating sums, multiplications, etc.) as it is a tutorial for understanding the game's strategic mechanics.

However, the following enemies will each have specially designed problems, also following the material taught in the ESO (Secondary Secondary Education) in consecutive terms. For example, taking into account the ESO material in general: Algebra, equations, functions, and systems of equations come first; then trigonometry and geometry make up the middle of the syllabus, and finally statistics and probability are the last. Similarly, the first four levels (including the special level) will be Algebra, equations, and functions, respectively; the next three will be geometry and trigonometry, and the last two will be statistics and probability, while the special level below encompasses statistics, probability, geometry, and trigonometry.

The final boss will have a pile of the entire syllabus with a slightly more difficult level, similar to the PAU (University of Puerto Rico) final exam, but at a lower level (around that level). Where everything learned so far will have to be applied.

2.5 Art and visual style

2.5.1 Artistic direction

The art style of this game will be low-poly with flat textures (and even textureless models with only the edges having volume), associating simple functions drawn in a notebook with the virtual world of Real Coordinate Space. Below are some artistic references that give a rough idea of what I'm trying to achieve: a very simple and schematic 3D low-poly art that will allow me to work primarily on the mathematical logic in the engine.



Figure 2.8: Artistic direction of Tactical Vector.

2.5.2 Scenery and enviorement

As it has been raised in Figure 2.8 , there will not be much color nor will the obstacles stand out so much, but enough to understand what it is, so that you can focus on the characters who will have contrasting colors while the stage models will have a contrast with a black border to differentiate them from the enemies and players.

2.5.3 UI y UX

Knowing more or less what the view in which you are going to play is like, in the game the UI will appear and disappear depending on whether it is your turn to move, your turn to attack, if you want to check your life, etc., more or less the layout of the game can be seen in Figure 2.9.

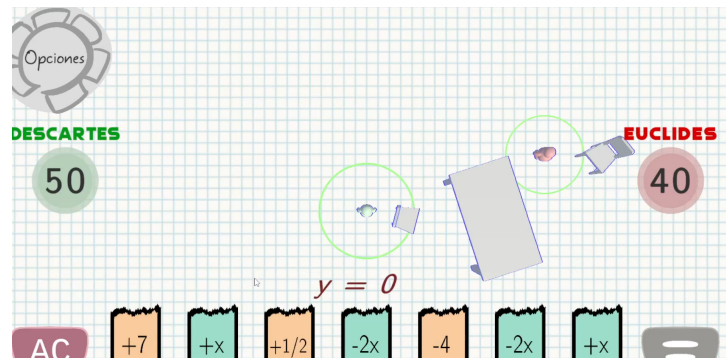


Figure 2.9: General user interface

Using the gear, you can access accessibility options to change the volume, return to the main menu, or exit to the map.

The main menu will display the game title and icon, a play option that will take you to the map, an options button that will take you to the aforementioned options, and a credits button that will take you to the credits screen (for the creator, supervisor, university, etc.).

2.6 Music and Sound

2.6.1 Soundtrack style

The goal is to achieve a style of music that is active but with little pressure, especially ambient but without thunderous power, rather in the background and above all causing a rather relaxing impact. An OST that I would love to simulate is Dr. Kawashima's Brain Training for Nintendo Switch, with a melody that is not very intrusive but active, so that you can see that the brain is working but has everything under control.

2.6.2 Key sound effects

Few actions will have sound effects. Managing menus and pressing buttons—I want them to have the same soft, unoriginal sound, like pressing a button in Fortnite, for example.

There's also scene switching, which will use the page-turning sound as if you were turning a page.

2.7 Technical aspects

2.7.1 Engine

Unity Engine 6000.0.45f1.

2.7.2 Programming languages

C#, base scripting language of Unity.

2.7.3 Minimal requisites and recommended

According to the official Unity 6 documentation (Manual: System requirements for Unity 6)¹, the requirements for mobile players in Unity builds are as follows:

Operating system	Operating system version	CPU	Graphics API	Additional requirements
Android	6.0 (API 23)+ Customized versions of Android must include all supported Google standard APIs.	ARMv7 with Neon Support (32-bit) or ARM64	OpenGL ES 3.0+, Vulkan	• 1GB+ RAM • Supported hardware devices must meet or exceed Google's Android Compatibility Definition (Version 9.0) limited to the following Device Types: 1. Handheld (Section 2.2) 2. Television (Section 2.3) 3. Tablets (Section 2.6) • Hardware must be running Android OS natively. Android within a container or emulator isn't supported. • For development: Android SDK (15/API 35), Android NDK (r27c) and OpenJDK (17), which are installed by default with Unity Hub.
iOS/iPadOS	13+	A8 SoC+	Metal	• Xcode version 15+ • For development and debugging: refer to Apple documentation on XCode support. • For App Store submission: refer to Apple's submission guidelines for the required Xcode version.

Figure 2.10: Requirements table according to the official Unity documentation.

¹Bibliography 4, [Unity n.d.](#)

2.7.4 AI and NPC behavior

AI will only be applied in combat, in which NPCs will always tend by default to search for a function that leaves them with the shortest possible distance between the player and themselves, considering the obstacles around them.

Then, when aiming, they will first look for features that intersect with the player's surface if they are within range; otherwise, they will not attack. If they find an intersection, they will check to see if there is an obstacle blocking the attack; if not, they will execute it.

They will follow that same loop until they are defeated.

CHAPTER 3

Technical implementation in Unity

3.1 Introduction

The educational video game *Tactical Vector* was developed using Unity, combining turn-based strategy mechanics with high school mathematics content focused on vectors. This chapter describes the game's technical implementation, divided into tasks ranging from initial setup to level design, art, UI, AI, and more.

3.2 Gameloop, UI y UX

3.2.1 Fuera de Combate

Menú Principal

At this point, the game will start loading into the main screen, which is the game's start menu. The title will appear, along with the Play, Options, Credits, and Exit buttons. Each of these buttons has a logical purpose related to its name:

- **Play**, It will load the level select minimap and the number of cards in your deck and unlocked perks you have (if you have them). This means that for each level, enemies will drop cards and, in special levels, passive buffs that will be automatically saved when you close the game. For more information on these values, check out the save system section.
- **Options**, will review aspects such as volume, go to the menu or the map.
- **Credits**, where the collaboration with the university, creator and all mentions of external licenses and elements used with their respective authors will be mentioned.
- **Exit** to exit the game.

Map

After defeating your first enemy and completing the tutorial, you'll be presented with a cutscene showing a map with various nodes and buttons to choose from. Selecting them will describe:

- The level number.
- Main topic (Algebra, systems of equations, etc.).
- Button to enter or exit that level.

All of these buttons will be enabled (and will not be visually less transparent) when a previously connected level has been completed.

In addition to selecting levels, the previous Options button is also available in the top left corner, with the additional option to exit the game. Note that, in this game, exiting the game will automatically save all progress made up to that point. Therefore, the exit button will be named "Save and Exit" instead of "Exit," as was previously stated in the main menu.

Another button is also available in the lower center corner, which is the Deck of Cards. Selecting this button will display an inventory of the cards prepared for the deck (with a minimum of 10 cards) in the upper half of the inventory, and in the lower half, the unused cards in the deck, which you can move from one list to another.

3.2.2 In Combat

Attack selection

Finally, upon entering combat and taking their turn, the player is able to choose up to 7 cards, which are available at the bottom center of the screen. Almost all combinations are possible, except for those that change x (see the Mathematical Foundation section for card types), in which only one of each type can be chosen per turn.

For each choice, slightly above, you'll see the formula you're creating ($\text{base } y = 0x + 0$) and a preview of it with two purple dots showing the two mobility options from the origin point at $x = 0$ where the player is positioned.

When you press the Send Formula button located slightly to the right of the card selection, two arrows will preview on the left and right, where you can choose which path to take: negative or positive. Finally, the player also has an options button at the top left of the screen in combat, essentially identical to the one on the map.

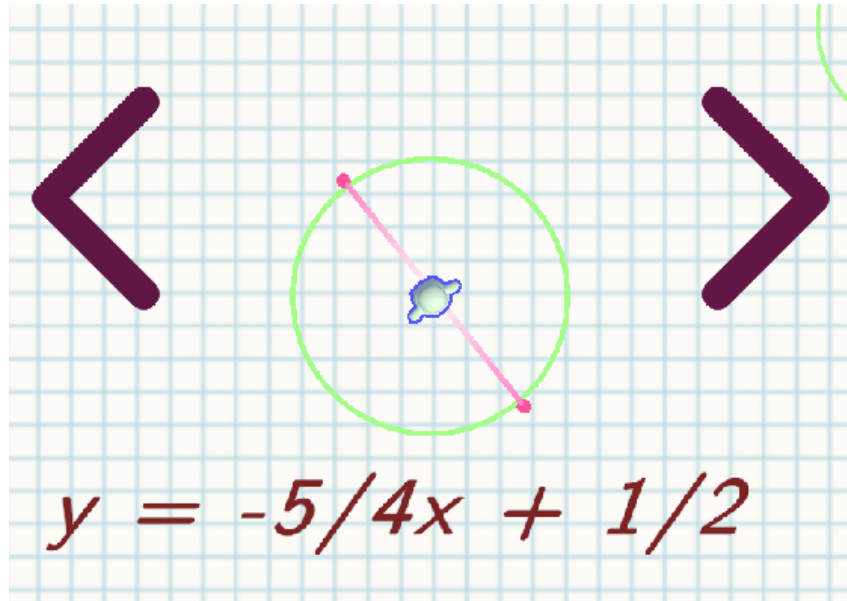


Figure 3.1: Movement to choose

3.3 Turn System

3.3.1 General Description

The way turn behavior is described is as mentioned in the design document: the player takes the first turn, then enemy 1, enemy 2, and back to the first enemy. To implement this, however, the movement and attack script used, as might be expected, are the same; therefore, the only change in the script it checks is the turn number assigned depending on the number of enemies in the specific level.

For example, if there are two enemies, the currentTurn integer value 0 applies the turn to the player, then the first enemy with a value of 1, and then the second enemy with a value of 2. The only changes are the camera focus and the application of the movement and attack formulas, using the current position values of the corresponding user.

3.3.2 Turn Controller

El script `GameManagerCombat.cs` handles the shift state from the moment it changes to the end of the update:

```
// Si es el turno finalizando de ataque va al siguiente enemigo

if (!isMoveState)
{
    if (targetTurn < enemyCount) targetTurn++;
    else targetTurn = 0;
}
```

A simple integer value is used for the turn order, also note that it is checked at the end of each turn if the state is not `isMoveState`, as that implies that it is in an attack state, therefore, at the end of the attack state the turn passes to the next in the queue, the first (number 0) being the player.

3.3.3 UI Integration

The UI updates with turn change events, especially when choosing a card. The visual formula appears below the player, and a preview is displayed from two nodes that execute the player's move for a short period of time, then become invisible at half the maximum movement radius.

By selecting the formula using the submit formula button, in most events (except for those where $x = \log(x)$, as x cannot be a negative value), you can choose the right or left option, either when attacking or moving, depending on what is convenient at the time.

Finally, note that there is also each player's health bar on screen and an options button to reduce the volume, change the resolution, exit, etc.

Here is a small snippet of code related to the UI, with, for example, how the formula values changed based on detecting whether the card highlight was green or black when the button was pressed:

```
if(activarCartas)
{
    for(int i = 0; i < SpriteCards.Length; i++)
    {
        if(SpriteCards[i].GetComponent<Button>().enabled ==
            false)
        {
            if(SpriteCards[i].GetComponent<RawImage>().color ==
                Color.black)
            {
                if(CartasMano[i].type == "change" && !hayChange)
                {
                    hayChange = true;
                    convertTo = CartasMano[i].convertTo;
                    SpriteCards[i].GetComponent<RawImage>().
                        color = Color.green;
                }
                else if(CartasMano[i].type == "var")
                {
                    m += CartasMano[i].value;
                    SpriteCards[i].GetComponent<RawImage>().
                        color = Color.green;
                }
                else if(CartasMano[i].type == "const")
                {
                    n += CartasMano[i].value;
                    SpriteCards[i].GetComponent<RawImage>().
                        color = Color.green;
                }
            }

            else if(SpriteCards[i].GetComponent<RawImage>().
                color == Color.green)
            {
                if(CartasMano[i].type == "change")
                {
                    hayChange = false;
                    convertTo = "";
                }
                else if(CartasMano[i].type == "var")
                {
                    m -= CartasMano[i].value;
                }
                else if(CartasMano[i].type == "const")
            }
```

```
        {
            n -= CartasMano[i].value;
        }
        SpriteCards[i].GetComponent<RawImage>().color =
            Color.black;
    }

    SpriteCards[i].GetComponent<Button>().enabled = true
    ;
}
}
```

3.4 Movement System and vectors

3.4.1 Mathematical fundament

Every mathematical move in the game is defined by the following simple formula:

$$y = mx + n \tag{3.1}$$

This formula, as is well understood, is identical to the function of the straight line in real coordinate space, where m is the slope and n is the displacement; however, it does not seek exactly the same representation. While formulas can be prepared that apply the equation of the straight line, it is very important to first understand how this formula is applied in the game itself.

Cards

To begin with, we will talk about **Cards** or, the ScriptableObject **Cards**, which are those selectable cards to add to your formula so $y = mx + n$. Examples of cards can be $+3x$, -7 , $+4$, $-5/2x$, $x = \sin(x)$... Each one saves its representative image, its identifying name, its value and its type.

There are 3 types of cards:

- **Constants**, which modify the variable displacement **n**.
- **Variables**, which modify the variable slope **m**.
- **Change**, which modify the expression of x. These are generally expressed as $x = \dots x$, since what they essentially do is give the proposed function the logarithmic, exponential, trigonometric, etc. effect, which simplifies the calculations needed personally to represent the function for the player and visualize it, instead of adding more complexity such as $y = \sin(x) + 7/3x - 2$, which would be more difficult to see.

These cards have the special feature that you cannot choose more than one in a turn.

So, one of the possible formulas to choose two sides that could be in Tactical Vector would be for example:

$$y = 3sen(x) - 2 \tag{3.2}$$

Where you could have taken a Constant -2 card, a Variable +3x card, and a Change $x = \sin(x)$ card.

Appliation in code

Originally, the way in which the mathematical formulas were going to work was to solve a system of equations between a circle centered at the origin (relative to the player) and any possible function generated by the sum of variables m and n and modification of x ($x = x^2, x = \sin(x)...$).

However, in more advanced systems that allow the possibility of more than 2 solutions or that simply have a way of clearing that is too complex, it was already a very extensive and unnecessarily complex application:

Listing 3.1: Primeros intentos para programar la fórmula de movimiento

```
/*  
  
ECUACION DE LA CIRCUNFERENCIA  
  
 $x^2 + y^2 = r^2 \longrightarrow y = \sqrt{r^2 - x^2}$  PARA SACAR LOS  
CORTES CON EL RADIO  
  
PRIMERA, ECUACION DE LA RECTA  
  
 $y = mx + n \longrightarrow$  Ecuacion de la recta, NOTAR QUE N TIENE QUE  
SER MENOR QUE EL VALOR RADIO ABSOLUTO.  
  
 $mx + n = \sqrt{r^2 - x^2}$  CORTES!!  
  
Primer corte  $\longrightarrow x1 = ( (-m * n + \text{Mathf.Sqrt}( (r * r) + (m * m) * (r * r) - (n * n) ) ) / ( (m * m) + 1 ) );$   
Segundo corte  $\longrightarrow x2 = ( - ( m * n + \text{Mathf.Sqrt}( (r * r) + (m * m) * (r * r) - (n * n) ) ) / ( (m * m) + 1 ) );$   
  
SEGUNDA, FUNCION CUADRATICA  
  
 $y = ax^2 + bx + c$   
  
 $ax^2 + bx + c = \sqrt{r^2 - x^2}$  CORTES!!  
  
La voy a simplificar (sino las lineas y calculos son  
desorbitados) para traer solo dos cortes  
  
 $mx^2 + n = \sqrt{r^2 - x^2}$ , NOTAR QUE N TIENE QUE SER MENOR  
QUE EL VALOR RADIO ABSOLUTO.  
  
Primer corte  $\longrightarrow x1 = \text{Mathf.Sqrt}(\text{Mathf.Sqrt}(4 * (m * m) * (r * r) + 4 * m * n + 1)/(m * m) - 1/(m * m) - (2 * n)/m)/\text{Mathf.Sqrt}(2)$   
Segundo corte  $\longrightarrow x2 = -\text{Mathf.Sqrt}(\text{Mathf.Sqrt}(4 * (m * m) * (r * r) + 4 * m * n + 1)/(m * m) - 1/(m * m) - (2 * n)/m)/\text{Mathf.Sqrt}(2)$   
  
 $y = m^{\wedge}x + n$   
  
 $m^{\wedge}x = \sqrt{r^2 - x^2}$   
 $(m^{\wedge}x)^2 = r^2 - x^2$   
  
*/
```

To simplify the application of this task, I took advantage of Unity's own logic by placing the entire radius surrounded by a spherical collider, so that when the character, guide or projectile leaves the radius they will directly stop, although this had the problem of causing complications when using exponential formulas in a very short time (since the change in values per frame would cause, due to the speed of the physics engine that had to be changed to continuous, otherwise they would stop in the next frame when they were outside the circumference, which was perhaps +5 away from it), it greatly simplified the application of formulas with slight complexity and generally made the code much more readable.

Listing 3.2: Mayor simplificación y aplicación del problema, por cada frame se aplica la fórmula.

```

else if (aplicarMovimiento)

    {
        if(isMoving)
        {
            if(m < 1.5f && m > -1.5f)
            x = x + (direccion) * (VELOCIDAD * 2);
            else
            x = x + (direccion) * VELOCIDAD;

            if(convertTo == "") y = m * x + n;
            else if(convertTo == "log") y = m * Mathf.Log10(
                x) + n;
            else if(convertTo == "sen") y = m * Mathf.Sin(x)
                + n;
            else if(convertTo == "^2") y = m * (x * x) + n;

            if(!isEnemyTurn)
            {

                if(isMoveState)
                avatarPosition.position = new Vector3(
                    temporalPosition.position.x + x, VELOCIDAD,
                    temporalPosition.position.z + y);
                else
                ataquePosition.position = new Vector3(
                    temporalPosition.position.x + x, 1,
                    temporalPosition.position.z + y);

            }

        }
    }
[...]
```

3.4.2 Map Movement

The general script of `GameManagerCombat.cs` directly calculates the movement and the attack also in the same way that the guide does, it relies on the collider named above called `OnHitColliderRadius.cs` to call the user who is applying said movement to stop, in the following code you can see that there is no difference between attack and movement, only the pointer and gameObject that move are the ones that change:

Listing 3.3: Código que detecta la colision del jugador para hacerlo parar.

```
using UnityEngine;
using System.Collections;

public class OnHitColliderRadius : MonoBehaviour
{
    public GameObject gameManagerCombat;

    public GameObject character;
    public GameObject attack;
    public GameObject possibilityLeft;
    public GameObject possibilityRight;

    void Start()
    {

    }

    void OnTriggerExit(Collider other)
    {
        if(other.gameObject == character) gameManagerCombat.
            GetComponent<GameManagerCombat>().isMoving = false;
        else if(other.gameObject == attack) gameManagerCombat.
            GetComponent<GameManagerCombat>().isMoving = false;
        else if(other.gameObject == possibilityLeft)
        {
            gameManagerCombat.GetComponent<GameManagerCombat>().
                isMovingA = false;
        }
        else if(other.gameObject == possibilityRight)
            gameManagerCombat.GetComponent<GameManagerCombat>().
                isMovingB = false;
    }
    [...]
}
```

So the script blocks the preview of attacks from left to right, the attack itself, and the player/enemy itself.

3.4.3 Camera Movement

Regarding the player's ability to view the map, since the game uses Cinema Machine as the basis of the project, the player can drag their finger across the screen to scroll and review the map, just as they would do with a top-down view in Google Maps. However, the map boundaries also limit camera movement, so you can't drag the camera further than the map's own limitations allow.

Additionally, when making a move or attacking, the camera will automatically lock onto the player or enemy, and after their turn, you can drag the camera again to review the space or be aware of your surroundings.

3.5 Combat and Math Problems

3.5.1 Exercise-based attacks

To attack, the player must solve an exercise depending on the current topic of the question, these are saved in `ScriptableObjects Exercises` where they are saved:

- Exercise ID.
- Problem statement.
- Image, if necessary for the problem statement.
- List of options, which can be between 4 and 6 (a...f).
- Damage value if it's a player attack.
- Damage value if it's an enemy attack.
- Bonus timer. If it runs out early, a bonus is given to the player.

All options are buttons that, when selected, will appear red or green to indicate a successful selection, depending on the exercise. The amount of damage subtracted or increased (in the case of an enemy attack) will depend on how many times you have failed the challenge. If you have the time bonus, the damage will be multiplied by 1.5, and if you solve an exercise and the enemy is attacking with the time bonus included, the enemy's attack will miss and will not reduce your health.

3.5.2 Problem Generator

The generator is a random number within a custom list for each enemy, meaning that the problems you encounter will vary depending on the enemy. And by facing the same enemy repeatedly, you'll increasingly learn their problems, taking less damage and defeating them faster.

3.6 AI Intelligence

All enemies share a common AI, which essentially focuses on getting closer to the player.

Enemies calculate three random combinations from their deck (which generally consists of no more than seven cards), and from these, calculating the final point, compare the other enemies' AI to find the closest point to the current player's position (using the Pythagorean theorem).

3.7 Level Design

3.7.1 Creating and Loading Levels

The levels are designed from a base scene (CombatTest), and from there, the game objects around the map that present obstacles (which have a collision system that prevents the player/enemy from passing through them) are duplicated and changed. More or less enemies are added. When creating them, the enemy values and references in the main script `GameManagerCombat`, which manages the turn cycle, relative positions, movement radii, among others, will also have to be modified.

This is why Unity itself loads the combat scenes and only externally loads the booleans for whether the player character has the radius and health perks and checks the `Deck` list to see what cards you have in your inventory. This is not done to enemies because, as mentioned above, the scene is specific to the specific combat and, therefore, the `EnemyDeck` and `Reward` are customized in the scene itself.

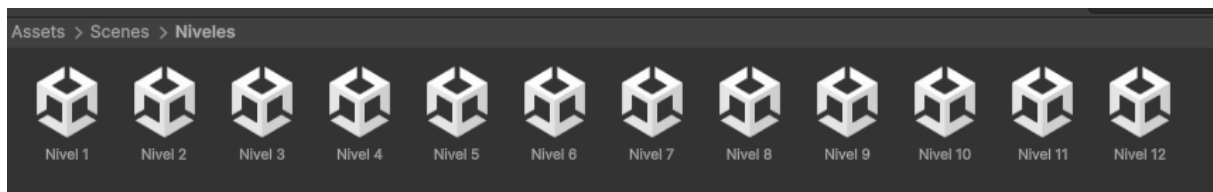


Figure 3.2: Previsualization of the icons of different scenes of levels.

3.8 Save system

For the save system, certain variables were taken into account, which had to always be available off-screen and, above all, could be saved in `PlayerPrefs`. The values to be saved were:

- List of card IDs in your deck.
- List of card IDs outside your deck but in your inventory.
- List of boolean numbers with completed levels.
- Boolean number that checks if you have the highest health perk.
- Boolean number that checks if you have the highest rank perk.

For the list, card identifier strings were saved individually, and then the identifiers were destroyed when loading them, essentially to save space.

3.9 Sound and Music

`AudioSource` and `AudioMixer` were used to manage downloaded music and effects, and these are also regulated using the `Volume` variable from the Audio Mixer shared by most of the Map, Combat, and Main Menu object options.

Essentially, all sounds in the game (including battle music and sound effects) are extracted and properly cited in the game in the credits of [WavSource n.d.](#), [Pixabay n.d.](#), and [Freesound n.d.](#). All sounds used have licenses that allow for academic use and expansion into a more ambitious project.

CHAPTER 4

Results

The results of the project were largely satisfactory, in the end the prototype planned for mobile devices was completed, with an extension of 12 levels and with the progression mechanics proposed in the technical proposal document, except for the upload of said prototype to the iOS and Android stores and the development of 3 different classes that were not the main one, therefore the only two characters present in the story were Hipparchus and Thue-Morse, while Laplace and Diophantus became the optional mini-bosses that gave special perks.

This was largely due to the large amount of time that was spent trying to make the classes have a different game feel between each one of them and the contradiction of adding problems or a higher percentage of special problems to certain specific characters playable by the player, since it is very possible that the players who most mastered x subject would always put on the same playable character knowing that they would have more opportunities to solve the exercises, perhaps ignoring the problems that would cost them the most, which is counterproductive for the game, which aims for all players to apply and study all types of subject they must learn, applying part of strategic positioning as a way of advantage for the player.

As for uploading the prototype to the Playstore and the iOS app page, it was canceled due to the lack of general playtesting with the target audience, which would be necessary to allow me to consider a product that can work for people in that sector.

CHAPTER 5

Conclusions and Future Lines of Work

In conclusion, this entire project has been a very formative experience, especially for me as a developer, as it has allowed me to once again specialize in the field of serious video games, which I began developing since my Erasmus at Hanze University of Applied Sciences. Above all, it has made me understand that, while it is necessary to continue developing the video game industry and its essence is entertainment, it doesn't just have to be pure entertainment, but also educational and, in my case, educational.

Furthermore, I was quite motivated when designing the project and I have been eager to refine it even with the problems that have arisen, mainly because mathematics is a subject that could be greatly promoted among young people in Spain, and I have always seen its potential in video games as being much more natural than many other subjects that could be taught (plus it has always been a branch of study that I have been passionate about learning).

Future lines of work would include, above all, more playtesting and surveys in collaboration with high schools around Castellón or the Valencian Community to refine certain aspects or add more content to the game. This could be a very useful tool, even for teachers, as they could be assigned to defeat a certain level or create challenges such as completing it with certain cards.

I would also like to refine the artistic aspect, especially regarding shaders. While I didn't aim to make them very complex in the project, a more visually pleasing game is the first thing consumers see, especially when selling a product that, by definition, isn't going to be as popular because it's about learning math.

Finally, I would like to work on expanding the number of levels and the number of exercises, since the stack must be larger so that the replayability is different each time, and include achievements in the game if possible, which would help the replayability of the levels (for example, passing levels without taking damage, or completing all the levels of the game), which above all would advocate exploiting the game's operation to the maximum for the intended audience.

Bibliography

1. I. N. de Evaluación Educativa, *TIMSS 2023. Estudio internacional de tendencias en matemáticas y ciencias. Informe español*. https://www.libreria.educacion.gob.es/libro/timss-2023-estudio-internacional-de-tendencias-en-matematicas-y-ciencias-informe-espanol_184942/, Último acceso el 2025-02-16.
2. I. N. de Evaluación Educativa, *RESULTADOS DE TIMSS 2023: INFORME ESPAÑOL*. <https://inee.educacion.es/2024/12/04/resultados-de-timss-2023-informe-espanol/>, Último acceso el 2025-02-16.
3. F. y. C. e. a. Fraga Varela, *Impacto de los juegos serios en la fluidez matemática: Un estudio en Educación Primaria*. https://www.researchgate.net/publication/354803398_The_impact_of_serious_games_in_mathematics_fluency_A_study_in_Primary_Education, Último acceso el 2025-02-17.
4. Unity, *System requirements for Unity 6.0, Unity Player system requirements for Mobile*, <https://docs.unity3d.com/6000.0/Documentation/Manual/system-requirements.html#mobile>, Último acceso el 2025-02-19.
5. WavSource, *WavSource.com*, <https://www.wavsource.com/sfx/sfx.htm>, Último acceso el 12 de Junio de 2025.
6. Pixabay, *Pixabay.com*, pixabay.com, Último acceso el 2025-06-12.
7. Freesound, *Freesound.org*, <https://freesound.org/>, Último acceso el 2025-06-12.

Appendix

APPENDIX A

Link to the code repository

The project's source code is managed by Github and can be accessed for preview and download through this link. The project requires, as mentioned in the report, Unity 6000.0.45f.

Tactical Vector Github Repository